

EtherCAT + M04 通讯学习笔记

从「为什么能通讯」到「工程每个文件是干什么的」——最近五轮问答精华整理

整理日期：2026-09-02 | 配套工程：test_M04 (SOEM + EC2-MB-M04)

本笔记导航

- 1. SOEM 库：哪来的？谁的？
- 2. “推” 状态机：M04 内部的状态升降机
- 3. 站号：M04 是 EtherCAT 的第几站？
- 4. 扫描后自动获得的信息：为什么 “这么匹配” ？
- 5. ecx_config_map_group 三个参数 & “10字节” 的真相
- 6. printSlaveInfo：是 M04 专属函数吗？数据哪来的？
- 7. test_M04 工程全景：四类文件各司其职
- 8. ESI/XML 文件：发挥什么作用？
- 9. 一课一测：快速自检

1. SOEM 库：哪来的？谁的？

SOEM = **S**imple **O**pen **E**therCAT **M**aster（开源 EtherCAT 主站库）。

来源结论：既不是自己写的，也不是 M04 官网提供的——是开源社区的项目，从网上下载的免费库。

版权方：Arthur Ketels（2005-2025）与瑞典 RT-Labs AB（2009-2025），可从 GitHub 免费获取。

对比项	你自己写的代码	SOEM 库
文件	test_m04.cpp（约190行，唯一业务代码）	约30个 .c 文件、几万行
角色	“指挥官” —— 决定做什么	“工具箱” —— 负责实现 EtherCAT 协议细节

类比	你决定用扳手拧哪个螺丝	买来的扳工具箱，不用自己造扳手
----	-------------	-----------------

SOEM 库内部结构（你不需要读懂它，但要知道它有哪些部分）：

```
soem/
├─ src/      ← 9个源文件（ec_config.c、ec_main.c... 库的“员工”）
├─ osal/     ← 操作系统适配层（Windows / Linux 各一套）
├─ oshw/     ← 网卡硬件接口（Windows 上调用 Npcap 抓包）
├─ include/  ← 头文件（soem.h 等“说明书”）
└─ samples/  ← 官方示例程序（printSlaveInfo 就是参考这里的写法）
```

提示：ethercat_win 和 test_M04 里的 SOEM 是**同一个库**（测试工程是从主工程复制了一份做隔离测试），所以两个工程的通讯底层一模一样。

2. “推” 状态机：M04 内部的状态升降机

“推” 是我用的通俗说法，完整表达是“**推状态机**”——**命令 M04 切换到某个工作状态。**

EtherCAT 从站有 4 个状态，像一台**只有 4 层、必须逐层上升的升降机**：

INIT (第1层 刚上电) → PREOP (第2层 预运行) → SAFE-OP (第3层 试运行) → **OP (第4层 干活状态)**

状态值：0x01 → 0x02 → 0x04 → 0x08

“推” 在代码里做了两件事（每一层都重复这两件事）：

① **提出要求**——告诉 M04 “请进入XX状态”，并把要求发出去：

```
ctx.slavelist[0].state = EC_STATE_SAFE_OP;    // 设定目标楼层（要进SAFE-OP）
ecx_writestate(&ctx, 0);                      // 按下按钮：发出切换指令
```

② **反复确认**——不停地发数据帧问“你到了吗”，直到 M04 回答“到了”才停：

```
do {
    ecx_send_processdata(&ctx);                // 发一帧数据（拍门问话）
    ecx_receive_processdata(&ctx, ...);         // 收回复
    ecx_statecheck(&ctx, 0, EC_STATE_SAFE_OP, ...); // 查状态：到了没？
} while (还没到 且 没超时);                    // 没到就继续问
```

为什么不能直接“瞬移”到 OP？ 每一层都要检查：链路通不通、参数配没配好、数据格式对不对——这是 EtherCAT 的**安全设计**。

你的测试程序推了两次，对应你看到的输出：

输出文字	含义
Slaves entered SAFE-OP state!	第一次推：从第1层爬到第3层 ✓
Slaves entered OP state!	第二次推：从第3层爬到第4层 ✓ ← 看到这句=通讯成功

3. 站号：M04 是 EtherCAT 的第几站？

当前测试环境：M04 是第 1 站（站号 1）。证据：程序输出 `Found 1 slave(s)`，且打印循环从 `cnt = 1` 开始——EtherCAT 的站号**从 1 开始**（0 保留给主站自己）。

站号怎么定？按物理连接顺序，谁离主站（电脑网卡）最近谁就是第 1 站：

电脑 → [设备A] → [设备B] → [设备C]
第1站 第2站 第3站

将来把 M04 接进主工程（3个TSJA20 + M04，去掉3个鸣志电机）：

电脑 → [TSJA20-1] → [TSJA20-2] → [TSJA20-3] → [M04]
第1站 第2站 第3站 第4站 ← M04变成第4站

这就是建议“M04 放最后一站”的原因：前 3 个电机的站号不变，主工程里“前3个从站=旋转电机”的代码逻辑就**不用大改**。

4. 扫描后自动获得的信息：为什么“这么匹配”？

运行程序时，一行行信息自动打印出来（名字、厂商ID 0xB95、产品代码 0x1410、输入输出字节数.....）。这些不是程序“猜”的，也不是“写死”的——是 M04 自己报上来的！

- ① M04 出厂时，艾莫迅把身份信息烧进它的 EEPROM 芯片（相当于**身份证**）
- ② 程序调用 ecx_config_init() 扫描 → SOEM 按协议标准向 M04 发“上交身份证”指令
- ③ M04 把 EEPROM 内容发给主站 → SOEM 解析后存进 ctx.slavelist[1]
- ④ 打印函数只负责把存好的数据显示出来

为什么信息“这么规整、正好匹配”？ 因为 EtherCAT 是**行业标准**：每个从站 EEPROM 的**固定位置**必须存厂商ID、固定位置必须存产品代码.....全世界的从站厂家都按这个标准烧录。程序按标准去固定位置“读取”，读出来自然是设备自己的真实身份。

扫描后拿到的字段	含义	M04 的值（举例）
name	设备名字	EC2-MB-M04
eep_man	厂商ID（身份证上的“发证机关”）	0xB95（艾莫迅）
eep_id	产品代码	0x1410
eep_rev	版本号	0x10100
Obytes / lbytes	输出/输入字节数（数据区大小）	输出 10 字节 / 输入看实际配置

类比：像收银台扫码——商品信息是商品自己带的，扫码枪只是读取并显示。M04 的 EEPROM=商品条码，SOEM=扫码枪，打印函数=收银显示器。

5. ecx_config_map_group 三个参数 & “10字节”的真相

关键一行：`ecx_config_map_group(&ctx, IOmap, 0);`

参数	名字	是什么	类比
① &ctx	上下文	SOEM 的“总档案柜”，保存所有从站信息（扫描结果都在里面）	全公司的员工档案
② IOmap	IO 映射缓冲区	一块 4096 字节的内存，SOEM 按每个从站的声明在这里“分格子”	数据仓库

③ 0	组号	第 0 组（把从站分组管理用；只有一串从站就填 0）	分组编号
-----	----	----------------------------	------

这个函数干的事：根据扫描到的“每个从站要几字节”，在 IOmap 仓库里划分区域，并把 `slavelist[1].outputs`（输出区指针）和 `slavelist[1].inputs`（输入区指针）自动指到对应格子——**你不用自己记地址。**

容易混淆的点：参数里的“0” ≠ “10字节”！

- 第三个参数 **0** = 组号（第0组，与大小无关）
- **10 字节** = M04 的输出区大小（Obytes），来自 M04 的 PDO 定义（Modbus Enable 1 字节 + Control Byte1~8 共8字节 + 对齐填充1字节 = 10字节）
- 验证方法：运行程序看 `Output PDO Size: 10 bytes` —— 那是 M04 自己声明的

6. printSlaveInfo: 是 M04 专属函数吗？数据哪来的？

不是 M04 专属！ 它是“通用从站信息打印器”，任何从站（TSJA20、鸣志电机）都能用。函数本身是我们参照 SOEM 官方样例（slaveinfo）写的——本质只是 9 行 printf 打印语句，**没有任何“发明数据”的能力。**

代码部分	性质
打印 Name / Vendor ID / Product Code / State / lbytes / Obytes...	通用： 每个从站都有这些字段，只是值不同
<pre>if (eep_man == 0xB95 && eep_id == 0x1410)</pre>	M04 专属： 唯一写死“期望值”的地方，作用是“验明正身”（检查读到的值是不是艾莫迅M04）

核心理解：程序里从头到尾没有“设置” M04 是 0xB95——M04 本来就是这个值。if 判断是拿读到的值**对比**预期值，读到了就标注“这是艾莫迅设备”。

7. test_M04 工程全景：四类文件各司其职

test_M04/	
├─ soem/	① SOEM开源库（第三方，占代码量99%）
├─ test_m04.cpp	② 业务代码（唯一，约190行）
├─ CMakeLists.txt	② 构建脚本（怎么编译+链接soem）
├─ CMakePresets.json	② VS生成器配置
├─ README.md	② 说明文档
├─ create_admin_shortcut.ps1	② 快捷方式辅助脚本
├─ AMX_EC2_MB_M04_....xml	③ M04官方ESI文件（设备描述）
├─ build/ out/ .vs/	④ 编译产物与VS缓存（自动生成，可删除）
└─ .git .gitignore	④ 版本控制文件

类别	内容	来源	你需要动吗？
① SOEM 库	soem/ 整个文件夹	开源社区下载（从 ethercat_win 复制）	不用动，当工具箱用
② 自己/辅助代码	cpp + 配置文件等	测试工程阶段由助手创建	真正要理解改动的就 test_m04.cpp

③ ESI/XML	AMX_EC2_MB_M04_general_0.1_1.0.xml	M04 官方（艾莫迅）发布	当资料查/以后装TwinCAT用
④ 自动生成	build/ .vs/ 等	CMake / VS / git 生成	不用管，可随时删除重建

8. ESI/XML 文件：发挥什么作用？

ESI = EtherCAT Slave Information (EtherCAT 从站信息文件) = M04 的 “**完整版说明书**”，由艾莫迅官方发布。里面写了：身份信息（厂商ID 0xB95、产品代码 0x1410）、对象字典（0x7000 Modbus Enable、Control Byte1~8）、PDO 定义（0x1600 输出10字节、0x1A00 输入）、DC 时钟配置、支持的 Modbus 功能码模块等。

关键：给谁用？两种主站玩法完全不同！

TwinCAT 类图形化主站		SOEM 类代码主站（你的工程）
认设备方式	扫描到不认识的新设备 → 查“从站库” → 必须手动安装XML ，否则不认识 M04	直接“问设备本人”：扫描时从 EEPROM 读取真实信息
XML 角色	必需（像装驱动/身份证复印件）	完全不需要 （代码和编译都不引用它）
你的 XML 实际用处	以后装了 TwinCAT 要安装它	当“参考手册”查 PDO/对象字典含义

为什么 EEPROM 够用还要 XML？ 因为 M04 的 EEPROM 只有 2KB，装不下完整说明书（XML 有几万行）。

- EEPROM（小纸条）：你是谁、PDO 多大等精简关键信息 —— SOEM 只信这个
- XML（大书）：完整对象字典、每个字节含义、图标、模块选项 —— TwinCAT 靠这个补齐细节

9. 一课一测：快速自检

看看你是否掌握了这次的内容——**先在心里作答，再看答案：**

#	问题	答案要点
1	SOEM 是 M04 官网提供的吗？	不是，是开源库（RT-Labs 等维护），免费下载
2	“推”状态机是什么意思？	命令从站逐级升状态：INIT→PREOP→SAFE-OP→OP，每级“提要求+反复确认”
3	当前测试 M04 是第几站？将来呢？	现在是第1站；放3个TSJA20后面则是第4站（站号=接线顺序）
4	打印的厂商ID等信息哪来的？	M04 出厂烧在 EEPROM，扫描时自己上报，程序只读取显示
5	ecx_config_map_group 第3个参数 0 是什么？	组号（第0组），不是字节数；“10字节”是M04输出区大小
6	printSlaveInfo 是 M04 专属吗？	不是，通用打印器；只有 if(0xB95) 判断是M04专属验证
7	工程里哪部分是第三方？哪部分是业务代码？	soem/=第三方库；业务代码只有 test_m04.cpp 等少量文件
8	XML(ESI) 在你的 SOEM 工程里被使用吗？	不被使用；是给 TwinCAT 认设备+给人查资料用的

终极一句话总结：通讯成功 = 网卡对 + M04在线 + 状态机“推”到OP；信息是M04自己报的；SOEM是别人的工具箱；XML是TwinCAT的说明书——而你真正要懂的代码，只有 test_m04.cpp。

本笔记由最近五轮问答整理而成，配套代码见 e:\JT-021\BEIJING_ECAT\test_M04\test_m04.cpp，设备描述见 AMX_EC2_MB_M04_general_0.1_1.0.xml。